Prepare — Harmonic, Associate SW Development Engineer (260140)

Context

This JD is a domain shift from your background: Harmonic wants embedded/networking systems work (Broadband Devices team), while your genuine experience is full-stack web/mobile/AI. Your honest overlap is C++ (CodeChef, algorithmic), Linux (daily Fedora use), and general debugging discipline. The gaps below are real, not just vocabulary — study them before an interview, don't overclaim depth you don't have.

MUST – core to the role

1. Layer 2/3 networking protocols

Layer 2 = Ethernet, MAC addressing, VLANs, switching. Layer 3 = IP addressing, routing, ARP, ICMP. One level below the HTTP/REST you already know. Bridge: you understand client-server communication already — the gap is one layer down (how a switch forwards frames vs. how a router forwards packets). A weekend of CompTIA Network+ level study covers the interview-safe 80%.

2. Embedded Systems

Programming for resource-constrained hardware: no/minimal OS, direct memory/register access, interrupt handling, no garbage collector, cross-compilation. This is your largest genuine gap — zero production experience. If pursued seriously: a basic C program on a Raspberry Pi or Arduino (GPIO blink, then UART echo) builds the "no runtime safety net" mental model your Node.js/Python background doesn't have. Be honest in interviews: "my C++ is algorithmic, not hardware-level" beats overclaiming.

3. Cross-layer debugging / root cause analysis

The JD wants debugging that spans layers (app symptom caused by a driver bug, hardware-dependent network issue) — not single-runtime debugging. Bridge: your real Sentry/DevTools debugging (reproduce, isolate, resolve) is genuine and transfers — the gap is layer, not method.

GOOD – nice to have

4. Perl

Older scripting language, common in networking/telecom tooling. Your Python fluency transfers quickly if needed — JD lists it as "advantage," not required.

5. Bash scripting depth

You already use Linux daily — verify depth (loops, functions, argument parsing) rather than deep-study from scratch.

Talking points

- Lead with CodeChef/DSA (6-star, 2200+, peak 2212; 1000+ problems) — your strongest literal match to "excellent at C++/C programming."
- Be upfront about the domain gap, framed as trajectory toward systems/embedded work.
- File Manager project (built twice: raw `node:http` vs. Express) is your closest honest bridge — shows the instinct to understand the layer below the abstraction.
- Your debugging methodology (reproduce, isolate, resolve) transfers even though the layers are unfamiliar — say so directly if asked.